

Memoriez™

Memoriez™ is a mobile application that allows users to document and revisit their travel memories through an interactive world map. Users can drop pins anywhere on the globe and attach photos, voice messages, and written notes to each location, creating a digital scrapbook anchored to real places.

The motivation is simple: travel memories are scattered. Photos sit in camera rolls, stories get buried in social feeds, and reflections are lost in notes apps, none of it tied to the places that made those moments meaningful. Existing solutions either prioritise broad social sharing at the cost of intimacy, or personal journalling at the cost of shareability. Memoriez™ bridges that gap.

The platform balances privacy with social connection. Users control the visibility of each memory, sharing selectively with friends who can like what they're given access to - personal enough to be a journal, social enough to be a shared experience.

Related Work

There are existing applications that address aspects of location-based memory keeping, but none of them fully combine the vision of a world map as the primary interface for attaching multimedia memories, with the ability to share memories and connect with others, while still retaining control over what stays private.

Polarsteps is the closest existing competitor. It automatically plots a user's path on a world map, and allows users to add photos, video, and text to each step that is recorded. Privacy controls let users decide who can see their trips. However, Polarsteps is fundamentally trip-oriented as memories are organised around different travel journeys rather than individual moments that can be pinned to any meaningful location. The social features, while present, are also asymmetric as followers can only passively observe a traveller's journey rather than contributing their own memories to shared places.

Day One is a journaling application with a map feature. It supports audio recording, unlimited photos and videos, and allows users to revisit past memories through a map view showing all locations from which entries were created. However, Day One's map is a secondary navigation tool, a way to find chronological journal entries rather than the primary interface for exploring and placing memories. The map acts as an index rather than a canvas. Social and sharing features are also minimal, as Day One is oriented entirely around private journaling with no meaningful way to share memories or connect with others.

Across these applications, there is a consistent gap; no existing mobile tool treats the world map as the primary canvas for multimedia memories at any freely pinned location, while also

offering meaningful social features that let users share and connect around the places that matter most to them. The proposed application addresses this gap directly.

Project Requirements

For the scope, we assume each team member is able to spend a total of 10 hours per week on this course. Taking the weekly two-hour lecture into consideration, there will be a total of 8 hours per member of development work.

This gives us a total development time of:

$$8 \times 7 = 56 \text{ hours per week}$$

At this stage, the project's scope will be defined through user stories and grouped by priority.

Must have:

1. As someone who wants to record my travels, I want to mark the locations I have been to so that I can visualise my travels.
2. As someone who wants to store my memories, I want to attach different forms of media to my memories so that I can look back and remember what happened.
3. As a person who wants to keep in touch with my friends, I want to be able to view my friends' memories.

Should have:

1. As someone who wants to travel, I want to be able to create a wishlist of locations so that I can view these at a later date and share them with my friends.
2. As someone who wants to discover new places, I want to be able to view others' memories so that I can get travel inspiration.
3. As someone who wants to share my photos, I want to be able to upload my pictures in bulk so that I can create memories faster.
4. As someone who wants to reminisce, I want to be alerted of memory anniversaries so that I never forget them.

Could have:

1. As someone who wants to remember my travels, I want to be able to create a digital souvenir of my memories so that I can symbolise my memories.
2. As someone who makes memories with friends, I want to be able to create shared memories with others so that I can look back at who I experienced a memory with.

Nice to have:

1. As a traveller, I want to export or share my visited places so I can view them on other platforms.
2. As a traveller, I want to be able to see how many countries I have been to so that I can stay motivated to visit more places.

Technologies

Mobile Framework — Expo (TypeScript)

Expo builds on React Native to simplify cross-platform development, providing a unified API for native device functionality on both iOS and Android.

Database — PostgreSQL (via Supabase)

PostgreSQL is a relational database that handles structured, user-related data more effectively than document-oriented alternatives like MongoDB. Its geospatial extensions via PostGIS align with our location-based requirements, and Supabase provides a convenient managed hosting environment.

ORM — Prisma

Prisma enables type-safe SQL queries from TypeScript and handles object-relational mapping, reducing boilerplate and minimising runtime database errors.

Backend — Node.js / Express

Express is a lightweight framework for building RESTful APIs that integrates seamlessly with our TypeScript stack.

User Authentication — Clerk

Clerk handles both authentication and authorisation out of the box, with pre-built flows for sign-up, sign-in, and session management, significantly reducing implementation overhead.

Project Management

Management Strategy

Drake Glazers uses a Scrumban approach, combining Scrum's sprint structure with Kanban's visual workflow management. Each task on the board is assigned to a team member with a due date. A Scrum Master role is assigned on a rotating basis each sprint, with a different team member taking on the responsibility each week. The Scrum Master is responsible for facilitating the sprint meetings, reviewing the backlog, keeping the team on track, and resolving any blockers.

Sprint Details

Sprints follow a one-week iteration cycle with two recurring meetings:

- **Sprint Assignment Meeting (Start of Week):** Tasks are delegated and relevant topics discussed.
- **Sprint Review Session (End of Week):** Blockers are addressed and outstanding items are considered for the next sprint.

Kanban Board Structure

The board has five columns representing workflow stages:

1. **To Do:** Entry point for all new work items. During the sprint assignment meeting, items are added with a title and brief description.
2. **Ready To Pick Up:** A team member expands the issue outside of meeting hours, adding acceptance criteria, a properly formatted title, and any other relevant detail. The issue is then moved here to signal it is ready to be picked up.
3. **In Progress:** Issues actively being worked on.
4. **Ready For Review:** Once work is complete, the assigned member opens a pull request and assigns a reviewer. If changes are required, the issue moves back to In Progress.
5. **Done:** Work has been reviewed, approved, and merged into main. The issue is formally closed.

Issue scope is kept small to ensure completion within a single sprint, leaving time for review and improvements.

Communication

The team meets twice a week for the sprint assignment and sprint review sessions, both facilitated by the Scrum Master for that week. Discord is the primary communication channel, with dedicated channels for general discussion, architecture, bug reports, and pull request reviews.

Version Control Strategy

The team follows GitHub Flow. Each issue falls into one of four categories: feature, bug fix, chore, or enhancement. For all categories, the process is:

1. Create a branch off main following the naming convention below.
2. Commit work regularly following the conventions below.
3. Open a pull request with an appropriate title and a short summary of changes in the description.
4. A team member not involved in the changes completes a code review.
5. The branch is merged into main and deleted.

Naming Conventions

Branch Names - [type]/[name]

- **Type:** bug, chore, feat, or improvement
- **Name:** 4–5 lowercase words, dashes instead of spaces

Commit Messages - [type]([scope]): [message]

- **Type:** bug, chore, feat, or improvement
- **Scope (optional):** Affected part of the app (e.g., auth, navbar, api)
- **Message:** Short description of the change

Pull Request Titles - [type]([scope]): [message] [issue number]

- Same fields as commit messages, plus the related issue number

To implement all the above, we will be using GitHub Projects as our centralised project management tool.

Risks In Team Projects

In any collaborative project, risks are situations that may negatively affect outcomes. Our team has identified and categorised key risks at the project's outset, following the recommended framework across four areas: Student Contribution, Team Self-Management, Resources, and Stakeholders. Mitigation strategies adhere to the PAME model and SRVA criteria.

Situation	Mitigation Plan	Monitoring	Action & Escalation
Availability & Communication: Scheduling conflicts or lack of communication	Meet twice weekly via Discord with a 24-hour response expectation for @mentions.	Attendance and responsiveness tracked via meeting minutes and Discord activity.	Action: Absent members review minutes and confirm tasks within 12 hours; direct contact if silent >24 hours. Escalate: No contact for 48 hours or repeated absence.
Process & Scope: Uneven workload, missed deadlines, or scope creep.	Use GitHub Projects with task estimation and ownership. Adhere strictly to Must/Should requirements; defer extras to backlog.	Team reviews board and sprint scope before meetings.	Action: Stalled tasks flagged immediately; new features deferred. Escalate: Must-Have delayed >48 hours or scope threatens delivery.

Expertise: Varying technical ability across teams.	Assign tasks with a primary owner and reviewer; enforce PR reviews for knowledge sharing.	PR reviews ensure code quality and shared understanding.	Action: Request help if stuck >50% of estimated time. Escalate: Unresolved technical blockers.
Co-ordination: Merge conflicts or integration issues.	Use strict branching with small, frequent commits.	Developers test and validate code before merging.	Action: Conflicts resolved by involved members. Escalate: Major codebase issues or loss of progress.
Stakeholders & Resources: Unclear requirements or external tool issues.	Clarify requirements before development; test third-party tools early. Maintain acceptance criteria and dependency tracking.	Requirements checked via tickets; dependencies tracked in shared doc.	Action: Clarify ambiguity within 24 hours; replace unsuitable tools. Escalate: Conflicting requirements or no viable alternatives.

Problem Escalation Process

Our team recognises that once an issue occurs, it requires a formal response. We will follow a structured Problem Process to evaluate whether an issue is within our control, attempting to resolve all internal conflicts first. We will only escalate to teaching staff after exhausting internal strategies, or when the root cause is outside our control (e.g., long-term illness or resource failure).